

Methods 3: Multilevel Statistical Modeling and Machine Learning

Week 44: Linear and logistic regression revisited (machine learning)
October 28, 2025

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Mid-way evaluation

TOTAL ANSWERS = 47

$n \geq 3$

Like	<i>count</i>
Teaching and Lectures	15
Practical sessions	12
Lecturer engaged and motivating	10
Relevance	8
Readings	5
Relation to Methods I & II	4
Live coding	4
Fast-paced lectures	4
Good amount of assignments	3
Code in slides	3
Balance between lecture and class	3

$$n < 3$$

Using the blackboard	2
Lectures are interactive	2
Good with math	2
Good explanations	2
Examples and visualisation	2
Assignments	2
Safe learning environment	1
Lecturer approachable	1
Lecture slides	1

$n \geq 3$

this time

DID NOT LIKE	<i>count</i>
Assignment interpretation	18
Fast paced	12
Too many assignments	7
Gelman book	5
Timing of handing in assignments	5
Setting up the conda environment	4
“New” questions in assignments	3
Assignments are hard	3
Methods 1 and 2 didn't prepare us	3

last time

Change	<i>count</i>
Too fast	9
Slides ahead of time	8
Math is hard to follow	6
More recommended reading/videos/coding	5
More practical instruction	3
Get RMD for my code	3

n<3

Stay on basic concepts	2
Relation to cognitive science industry	2
Simpler examples	2
Exam that promoted individual work	2
Slides with text instead of code	2
Practice live coding	1
Live coding too quick	1
Repetition exercises	1
Go over readings in lectures	1
Materials on brightspace	1
Spacing between assignments	1
Structure of lectures	1
Extra exercises	1
Improved slides	1
Knowing the assignment deadlines	1

Summary – what I promise to change

I will slow down the pace

I will *not* upload slides, however. (I want you to keep your attention on the lecture, not your notes)

What you should change – start asking questions!

- To slow me down – start asking questions!
 - You ask because you want to know, and I answer because I want to tell you

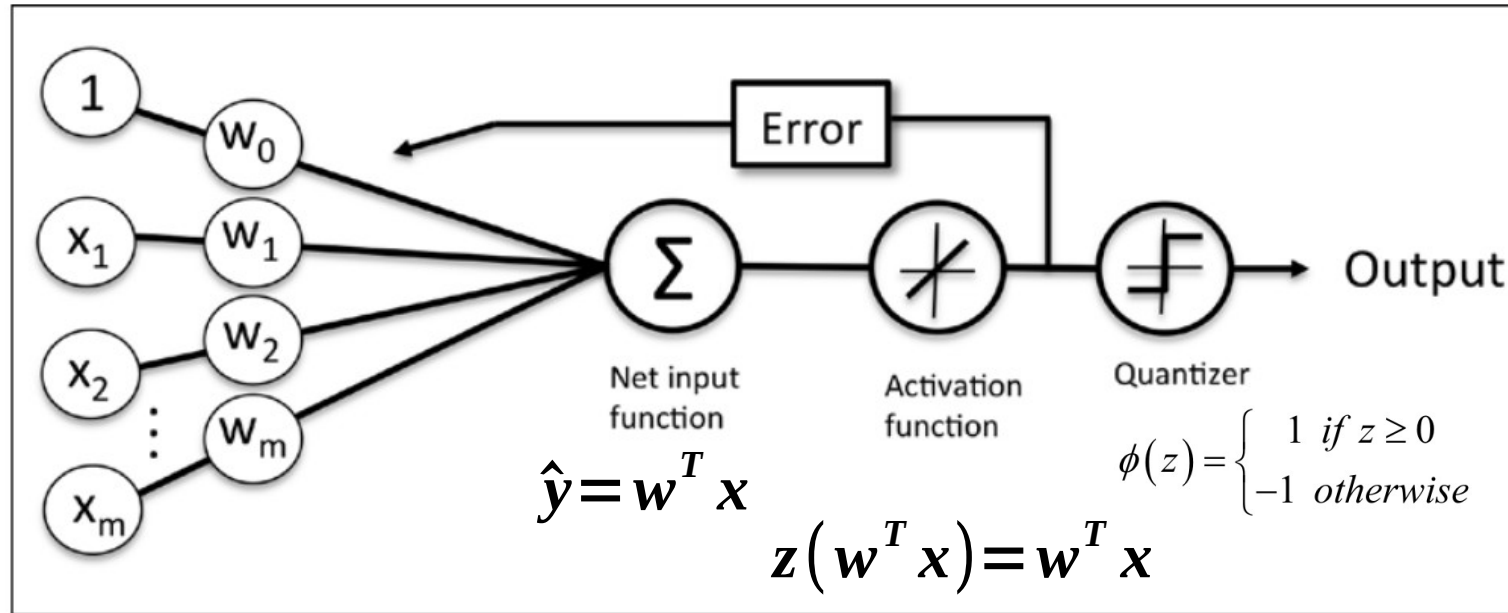
ADALINE

ADaptive LInear NEuron

DECODING

Raschka S (2015) Python Machine Learning. Packt Publishing Ltd

ADaptive Linear NEuron (ADALINE)



$$w^T x = w_0 x_0 + w_1 x_1 + \dots + w_{m-1} x_{m-1} + w_m x_m$$

ADALINE Gradient descent

```
def __init__(self, eta=0.01, n_iter=50):  
    self.eta = eta  
    self.n_iter = n_iter
```

```
class AdalineGD(object):  
    """ ADaptive LInear NEuron classifier  
  
    Parameters  
    -----  
    eta : float  
        Learning rate (between 0.0 and 1.0)  
    n_iter : int  
        Passes over the training dataset.  
  
    Attributes  
    -----  
    w_ : 1d-array  
        Weights after fitting.  
    errors_ : list  
        Number of misclassifications in every epoch.  
  
    """
```

Methods

$$\hat{y} = w^T x$$

$$z(w^T x) = w^T x$$

$$\phi(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

```
def net_input(self, X):  
    """Calculate net input"""  
    return np.dot(X, self.w_[1:]) + self.w_[0]  
  
def activation(self, X):  
    """Computer linear activation"""  
    return self.net_input(X)  
  
def predict(self, X):  
    """Return class label after unit step"""  
    return np.where(self.activation(X) >= 0.0, 1, -1)
```

The *fit* method

eta (η): learning rate (a constant)

output: $X\hat{\beta} = \hat{y}$

errors: $y - \hat{y}$

X.T.dot(errors): $X^T \cdot (y - \hat{y})$

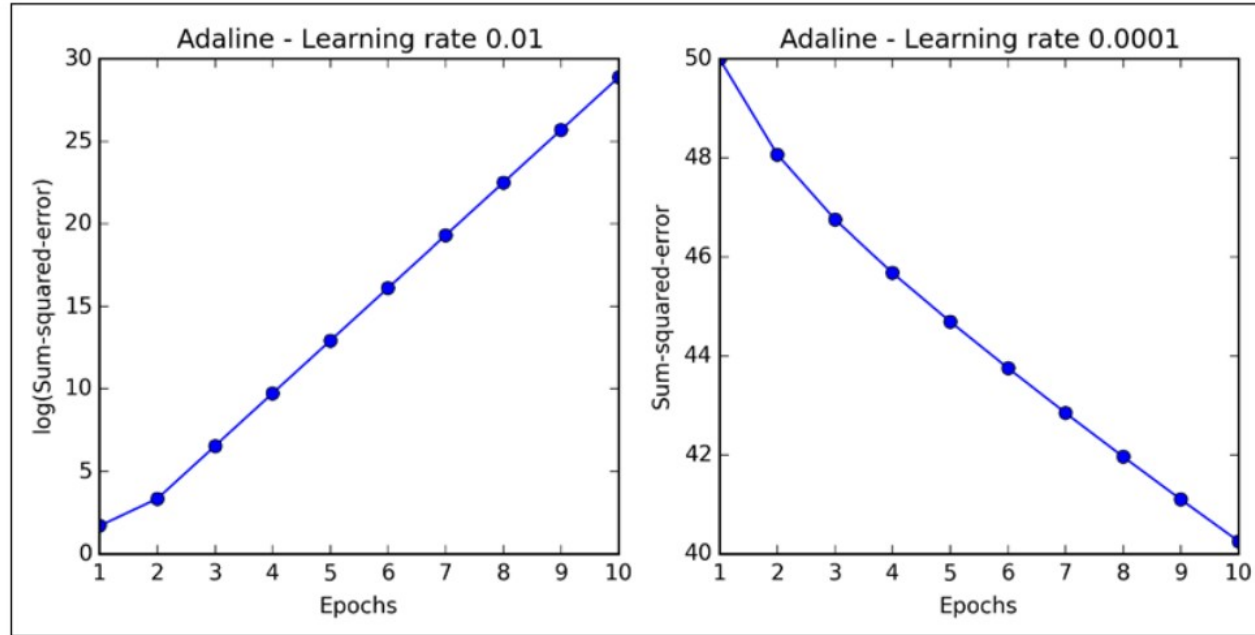
$X^T \cdot (y - \hat{y}) = \Delta w_1 + \Delta w_2 + \dots + \Delta w_{m-1} + \Delta w_m$

cost function: $(\sum (y - \hat{y})^2) / 2$

```
def fit(self, X, y):  
    """ Fit training data.  
  
    Parameters  
    -----  
    X : {array-like}, shape = [n_samples, n_features]  
        Training vectors, where n_samples  
        is the number of samples and  
        n_features is the number of features.  
    y : array-like, shape = [n_samples]  
        Target values.  
  
    Returns  
    -----  
    self : object  
  
    """  
    self.w_ = np.zeros(1 + X.shape[1])  
    self.cost_ = []  
  
    for i in range(self.n_iter):  
        output = self.net_input(X)  
        errors = (y - output)  
        self.w_[1:] += self.eta * X.T.dot(errors)  
        self.w_[0] += self.eta * errors.sum()  
        cost = (errors**2).sum() / 2.0  
        self.cost_.append(cost)  
    return self
```

Learning rate

Overshooting
the global
minimum

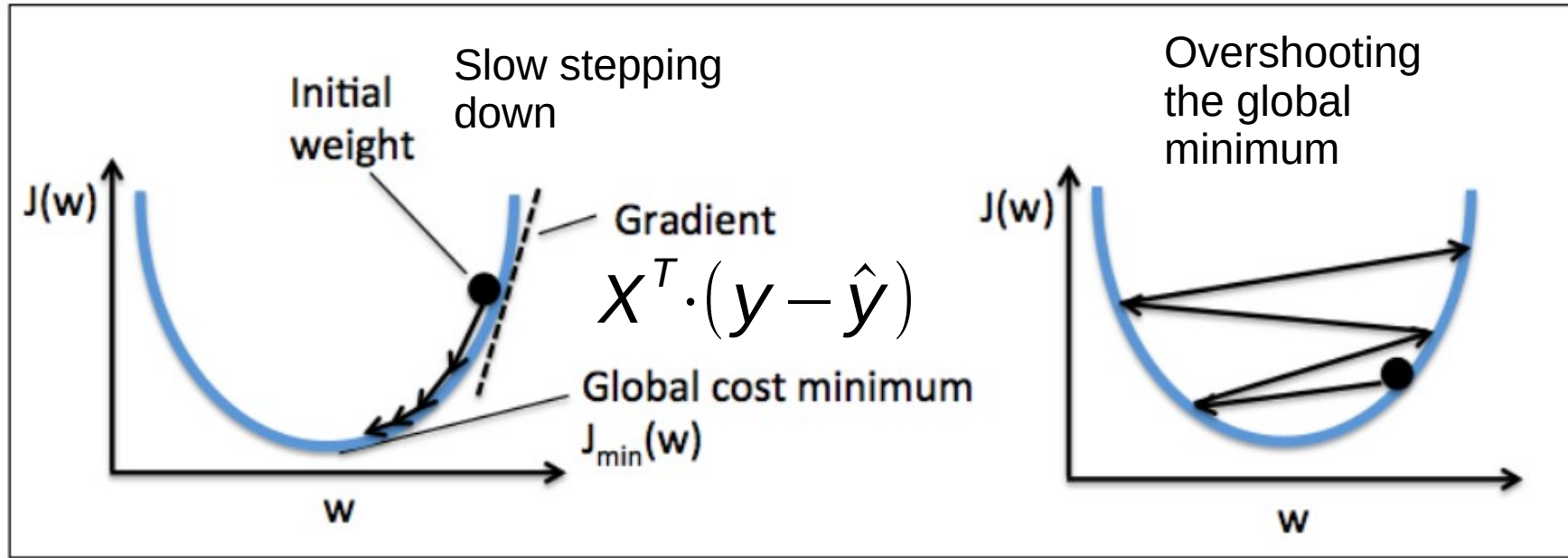


Slow stepping
down

Always check whether it converges

(p. 40: Raschka, 2015)

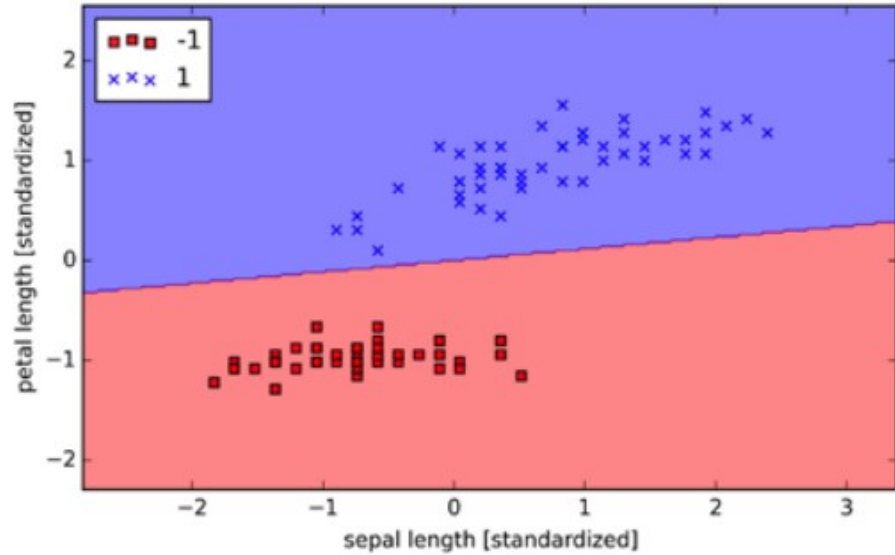
Gradient descent



$$J(w) = (\sum (y - \hat{y})^2) / 2$$

(p. 40: Raschka, 2015)

Adaline - Gradient Descent



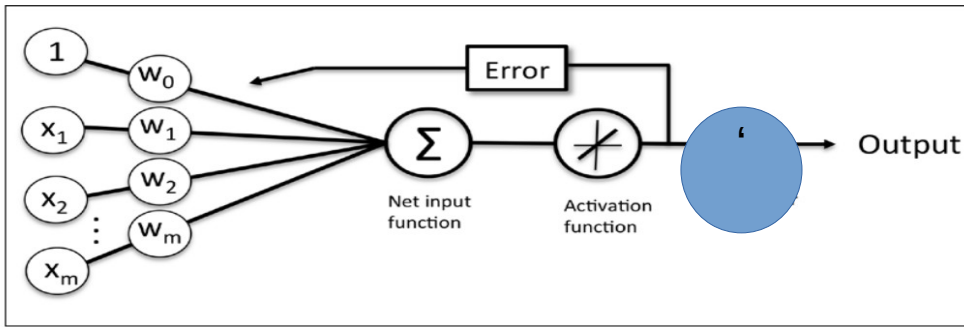
(p. 42: Raschka, 2015)

```
In [163]: ada.w_  
Out[163]: array([-0.40808285, -0.33924452,  0.79202224])
```

```
In [155]: X_subset[0, :]  
Out[155]: array([5.1, 1.4])
```

$$\hat{y}_1 \approx -0.408 - 0.339 \cdot 5.1 + 0.792 \cdot 1.4 = -1.03$$

```
In [164]: ada.net_input(X_subset[0, :])  
Out[164]: -1.029398778794785
```



How does this relate to linear regression?

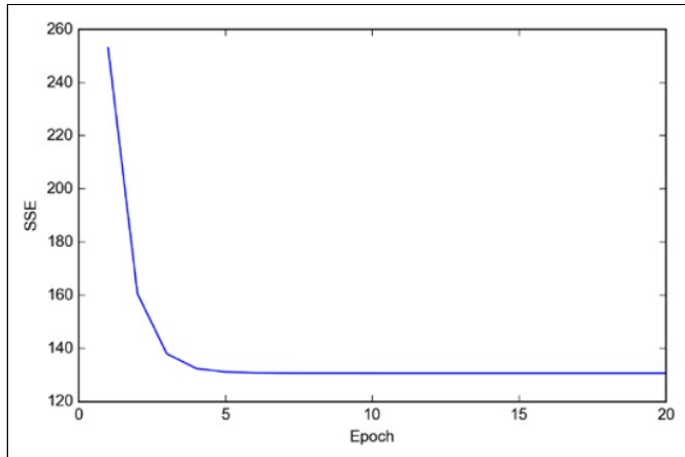
From ADALINE

$$\mathbf{w}^T \mathbf{x} = w_0 x_0 + w_1 x_1 + \dots + w_{m-1} x_{m-1} + w_m x_m$$

Very similar to ADALINE
and when converged will be virtually
identical to the ordinary least
squares solution

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Convergence



```
class LinearRegressionGD(object):

    def __init__(self, eta=0.001, n_iter=20):
        self.eta = eta
        self.n_iter = n_iter

    def fit(self, X, y):
        self.w_ = np.zeros(1 + X.shape[1])
        self.cost_ = []

        for i in range(self.n_iter):
            output = self.net_input(X)
            errors = (y - output)
            self.w_[1:] += self.eta * X.T.dot(errors)
            self.w_[0] += self.eta * errors.sum()
            cost = (errors**2).sum() / 2.0
            self.cost_.append(cost)
        return self

    def net_input(self, X):
        return np.dot(X, self.w_[1:]) + self.w_[0]

    def predict(self, X):
        return self.net_input(X)
```

ADALINE

```
def fit(self, X, y):
    """ Fit training data.

    Parameters
    -----
    X : {array-like}, shape = [n_samples, n_features]
        Traing vectors, where n_samples
        is the number of samples and
        n_features is the number of features.
    y : array-like, shape = [n_samples]
        Target values.

    Returns
    -----
    self : object

    """
    self.w_ = np.zeros(1 + X.shape[1])
    self.cost_ = []

    for i in range(self.n_iter):
        output = self.net_input(X)
        errors = (y - output)
        self.w_[1:] += self.eta * X.T.dot(errors)
        self.w_[0] += self.eta * errors.sum()
        cost = (errors**2).sum() / 2.0
        self.cost_.append(cost)
    return self
```

```
def net_input(self, X):
    """Calculate net input"""
    return np.dot(X, self.w_[1:]) + self.w_[0]

def activation(self, X):
    """Computer linear activation"""
    return self.net_input(X)

def predict(self, X):
    """Return class label after unit step"""
    return np.where(self.activation(X) >= 0.0, 1, -1)
```

```
class LinearRegressionGD(object):
```

```
    def __init__(self, eta=0.001, n_iter=20):
        self.eta = eta
        self.n_iter = n_iter

    def fit(self, X, y):
        self.w_ = np.zeros(1 + X.shape[1])
        self.cost_ = []

        for i in range(self.n_iter):
            output = self.net_input(X)
            errors = (y - output)
            self.w_[1:] += self.eta * X.T.dot(errors)
            self.w_[0] += self.eta * errors.sum()
            cost = (errors**2).sum() / 2.0
            self.cost_.append(cost)
        return self

    def net_input(self, X):
        return np.dot(X, self.w_[1:]) + self.w_[0]

    def predict(self, X):
        return self.net_input(X)
```

Live coding ADALINE

(*W43_live_coding.py*)

Interim summary

- We can build simple classification tools using scikit-learn
 - These can give us decision boundaries
 - That we can apply to new data (we haven't done that yet)
- We need to define cost functions
 - These can be conceptually separated from prediction functions (remember ADALINE)

LINEAR REGRESSION

The features of the 506 samples may be summarized as shown in the excerpt of the dataset description:

- **CRIM**: This is the per capita crime rate by town
- **ZN**: This is the proportion of residential land zoned for lots larger than 25,000 sq.ft.
- **INDUS**: This is the proportion of non-retail business acres per town
- **CHAS**: This is the Charles River dummy variable (this is equal to 1 if tract bounds river; 0 otherwise)
- **NOX**: This is the nitric oxides concentration (parts per 10 million)
- **RM**: This is the average number of rooms per dwelling
- **AGE**: This is the proportion of owner-occupied units built prior to 1940
- **DIS**: This is the weighted distances to five Boston employment centers
- **RAD**: This is the index of accessibility to radial highways
- **TAX**: This is the full-value property-tax rate per \$10,000
- **PTRATIO**: This is the pupil-teacher ratio by town
- **B**: This is calculated as $1000(B_k - 0.63)^2$, where B_k is the proportion of people of African American descent by town
- **LSTAT**: This is the percentage lower status of the population
- **MEDV**: This is the median value of owner-occupied homes in \$1000s

(p. 229: Raschka, 2015)

California Housing Dataset

x

MedInc: median income in block group

HouseAge: median house age in block group

AveRooms: average number of rooms per household

AveBedrms: average number of bedrooms per household

Population: block group population

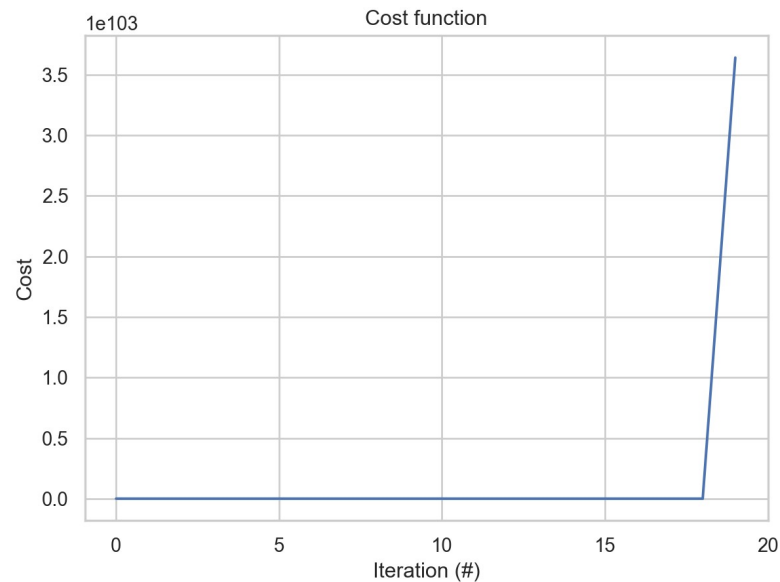
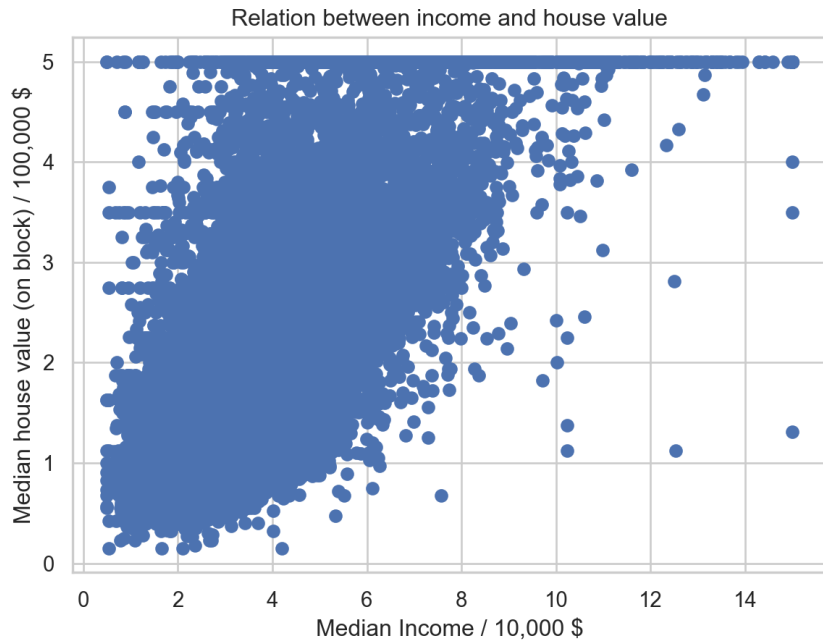
AveOccup: average number of household members

Latitude: block group latitude

Longitude: block group longitude

y

MedHouseVal: The median house value for California districts, expressed in hundreds of thousands of dollars (\$100,000)



```
LR = LinearRegressionGD()
LR.fit(X[:, 0:1], y) ## just fitting on Median Income
```

$$J(w) = (\sum (y - \hat{y})^2) / 2$$

```
print(LR.w_)
```

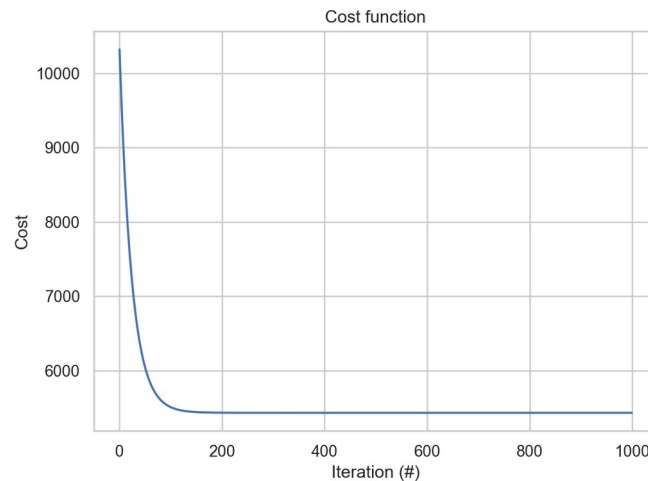
```
## [-1.10873027e+51 -5.27207344e+51]
```

!

Check for convergence!

```
X, y = fetch_california_housing(return_X_y=True)
from sklearn.preprocessing import StandardScaler
sc_x = StandardScaler()
sc_y = StandardScaler()
X_std = sc_x.fit_transform(X)
y_std = np.squeeze(sc_y.fit_transform(y.reshape(-1, 1)))

LR = LinearRegressionGD(eta=1e-6, n_iter=1000)
LR.fit(X_std[:, 0:1], y_std) ## just fitting on Median Income
```



```
class LinearRegressionGD(object):
    def __init__(self, eta=0.001, n_iter=20):
        self.eta = eta
        self.n_iter = n_iter
```

- Standardisation

$$x_{std}^{(i)} = \frac{x^{(i)} - \mu_x}{\sigma_x}$$

μ_x : sample mean for the feature: x

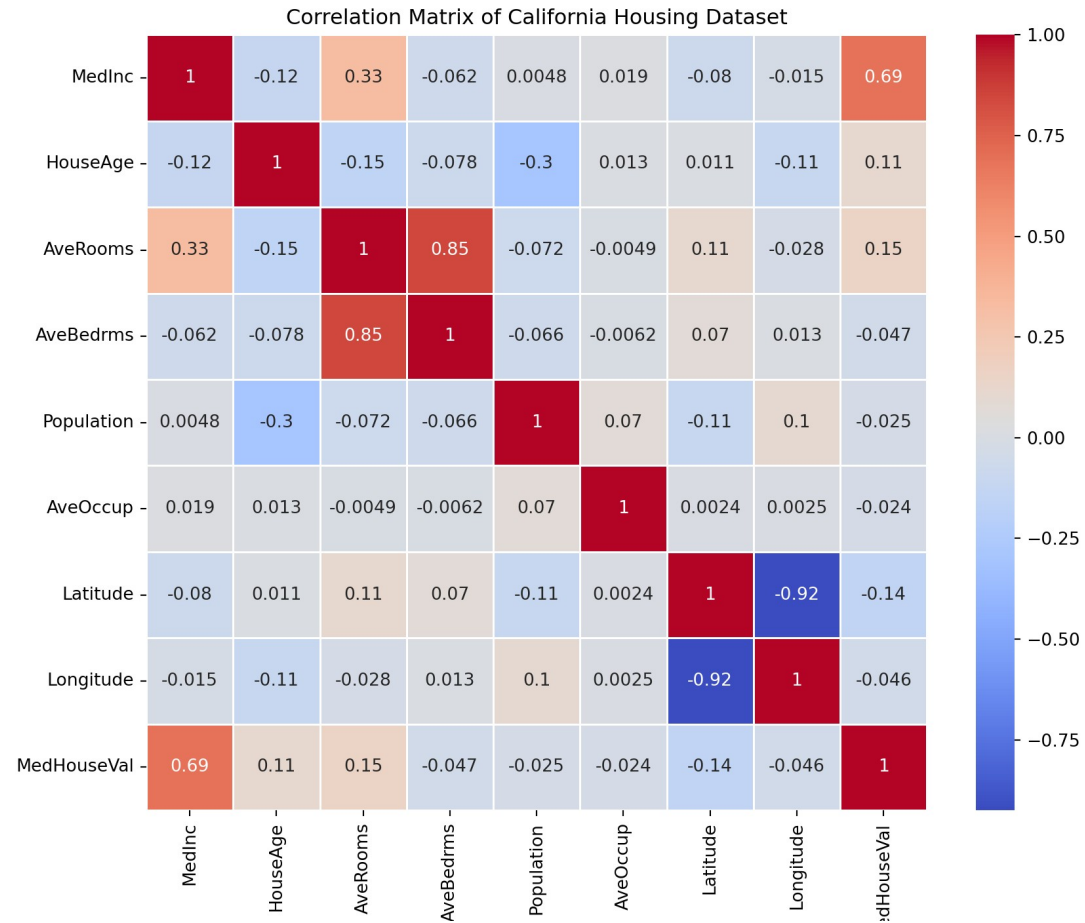
σ_x : sample standard deviation for the feature: x

Brings data onto a normal distribution with $\mu=0$ and $\sigma=1$



Multiple linear regression

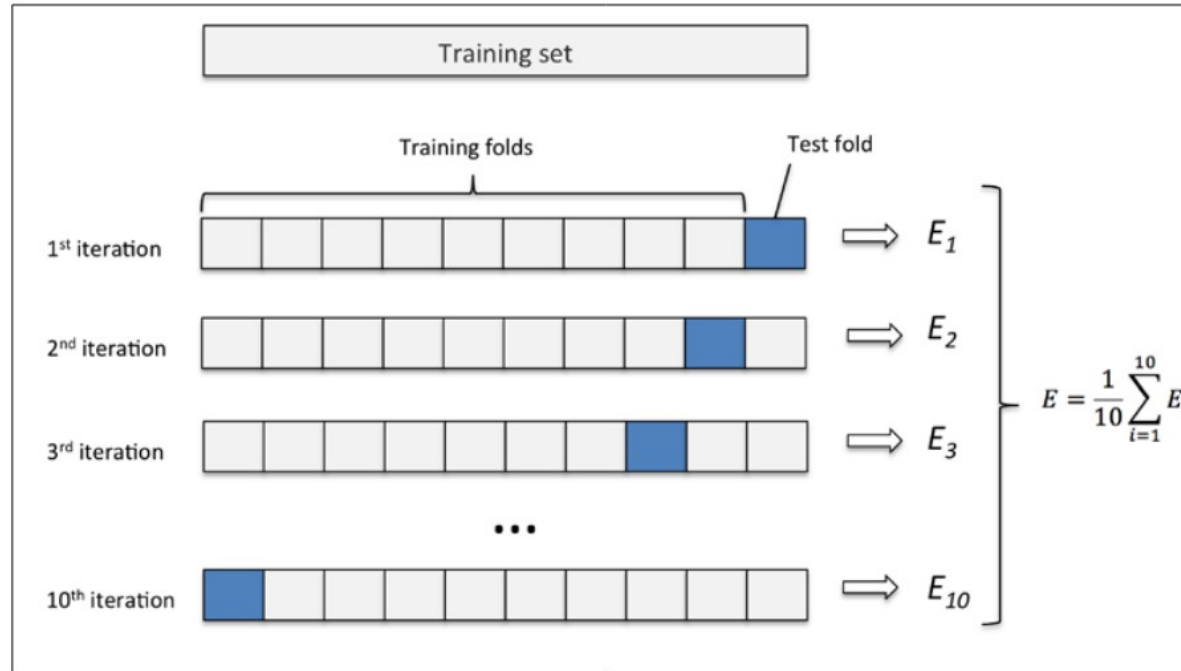
$$\begin{aligned} \hat{MedHouseVal} = & \hat{w}_0 x_0 + \hat{w}_1 MedInc + \hat{w}_2 HouseAge \\ & + \hat{w}_3 AveRooms + \hat{w}_4 AveBedrms \\ & + \hat{w}_5 Population + \hat{w}_6 AveOccup + \hat{w}_7 Latitude + \hat{w}_8 Longitude + \epsilon \end{aligned}$$



Because of the collinearity, we know we are prone to overfitting, so we do **out-of-sample** prediction instead of validating our model with traditional measures like R^2 and maximum likelihood

How to choose the **out-of-sample** dataset?

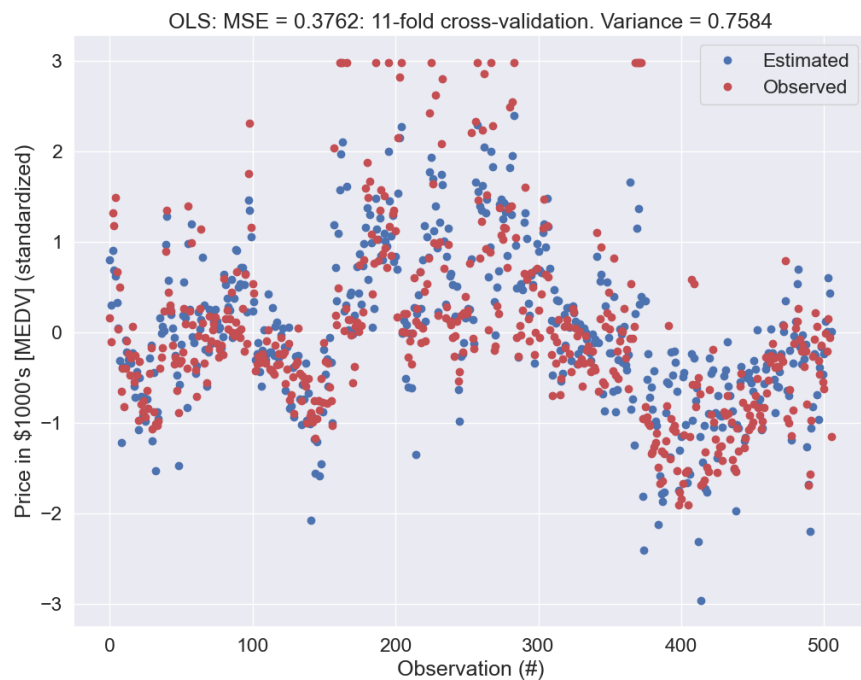
Cross-validation



(p. 176: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2025

```
OLS = LinearRegression()  
OLS.fit(X_std, y_std)  
  
MSE = np.mean(cross_validate(OLS, X_std, y_std, k=11))
```

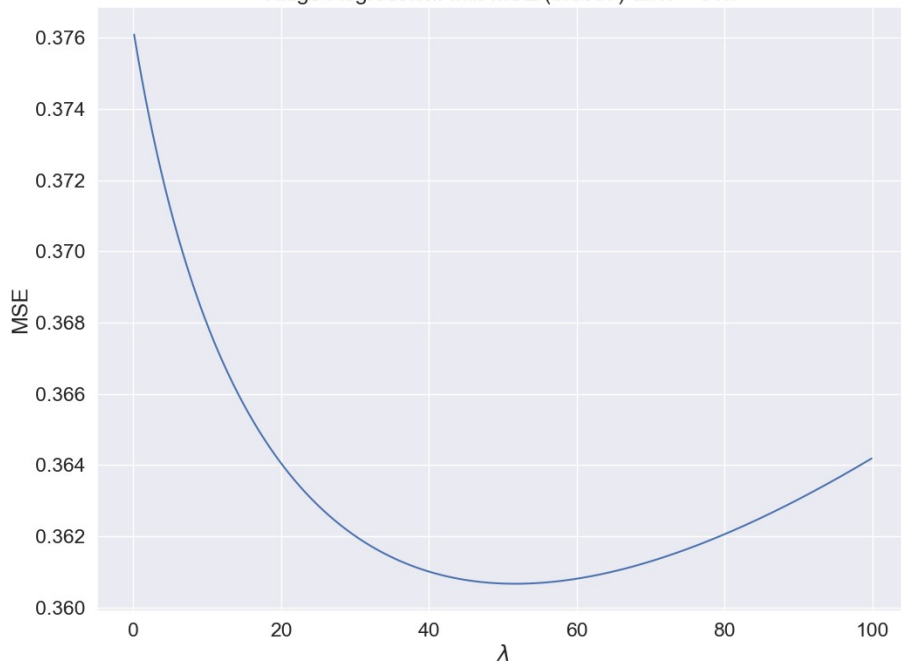


We can impose penalties

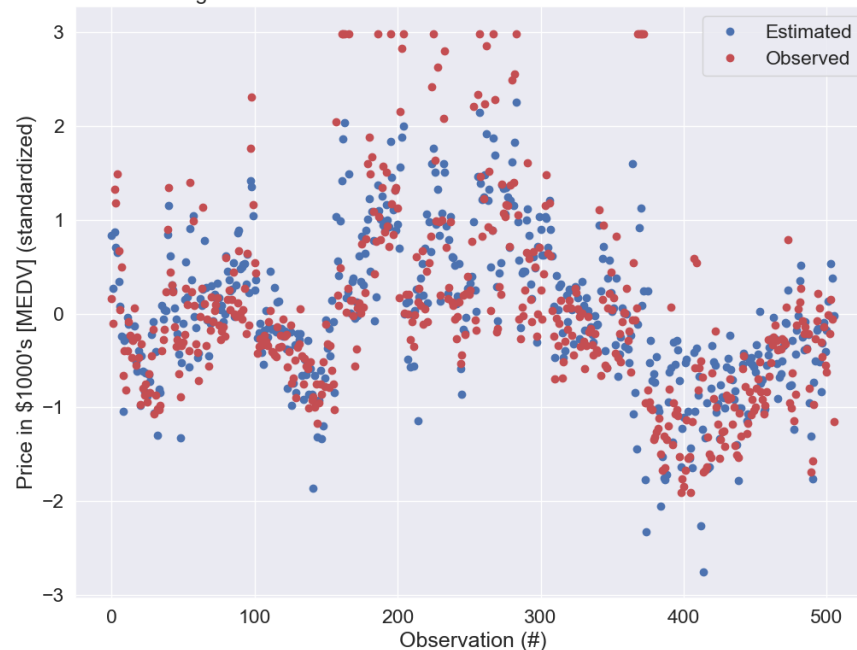
(but not on the intercept)

$$J(w)_{\text{Ridge}} = \sum^n \left(y^{(i)} - \hat{y}^{(i)} \right)^2 + \lambda \|w\|_2^2$$

Ridge Regression: min MSE (0.3607) at: $\lambda = 51.7$



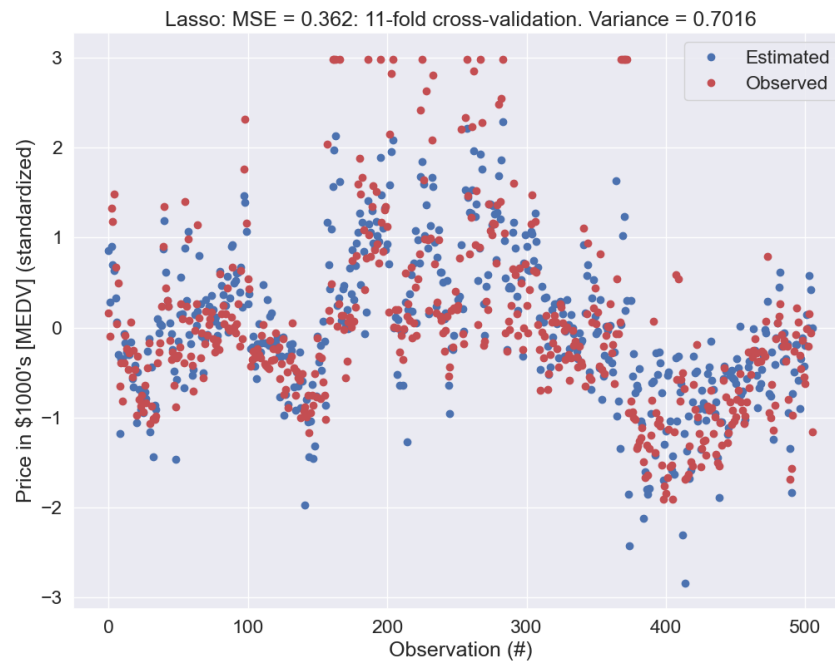
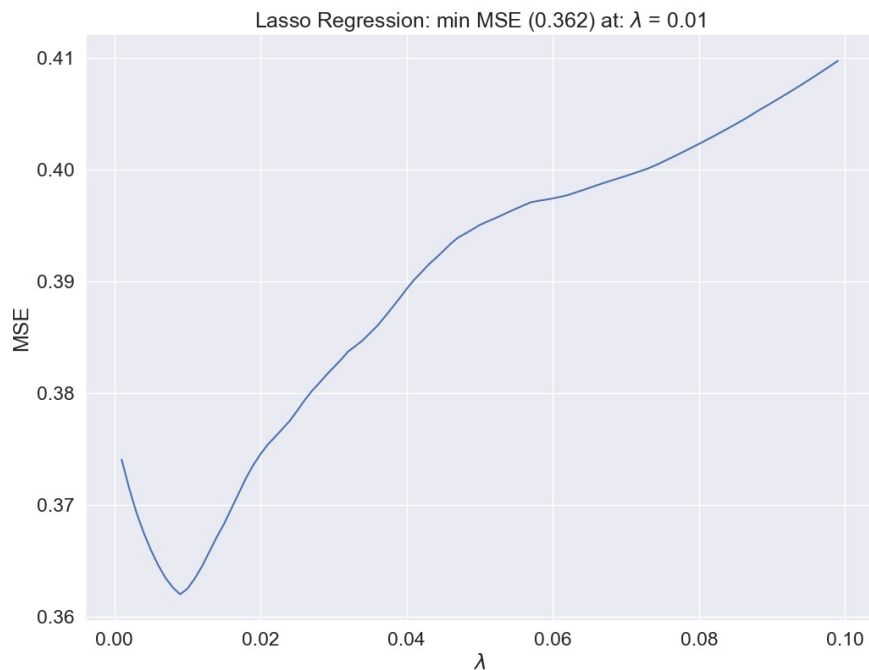
Ridge: MSE = 0.3607: 11-fold cross-validation. Variance = 0.6532



We can impose penalties

(but not on the intercept)

$$J(w)_{LASSO} = \sum_{i=1}^n \left(y^{(i)} - \hat{y}^{(i)} \right)^2 + \lambda \|w\|_1$$



Coefficients are shrunk

```
In [131]: OLS.coef_
```

```
Out[131]:
```

```
array([-0.09874812,  0.12473758,  0.02386168,  0.06945318, -0.22231612,  
        0.2911837 ,  0.0325356 , -0.32907266,  0.34212986, -0.28361575,  
       -0.21482076,  0.09763631, -0.43131412])
```

```
In [132]: RR.coef_
```

```
Out[132]:
```

```
array([-0.07536542,  0.08180161, -0.03182673,  0.07855868, -0.13185121,  
        0.31082552,  0.00548938, -0.23491485,  0.11242408, -0.0959682 ,  
       -0.18436614,  0.09154233, -0.36579723])
```

```
In [133]: lasso.coef_
```

```
Out[133]:
```

```
array([-0.07348948,  0.09107936, -0.          ,  0.07003181, -0.17110318,  
        0.30950805,  0.          , -0.29010247,  0.16456993, -0.1319311 ,  
       -0.19668957,  0.09063304, -0.41664587])
```

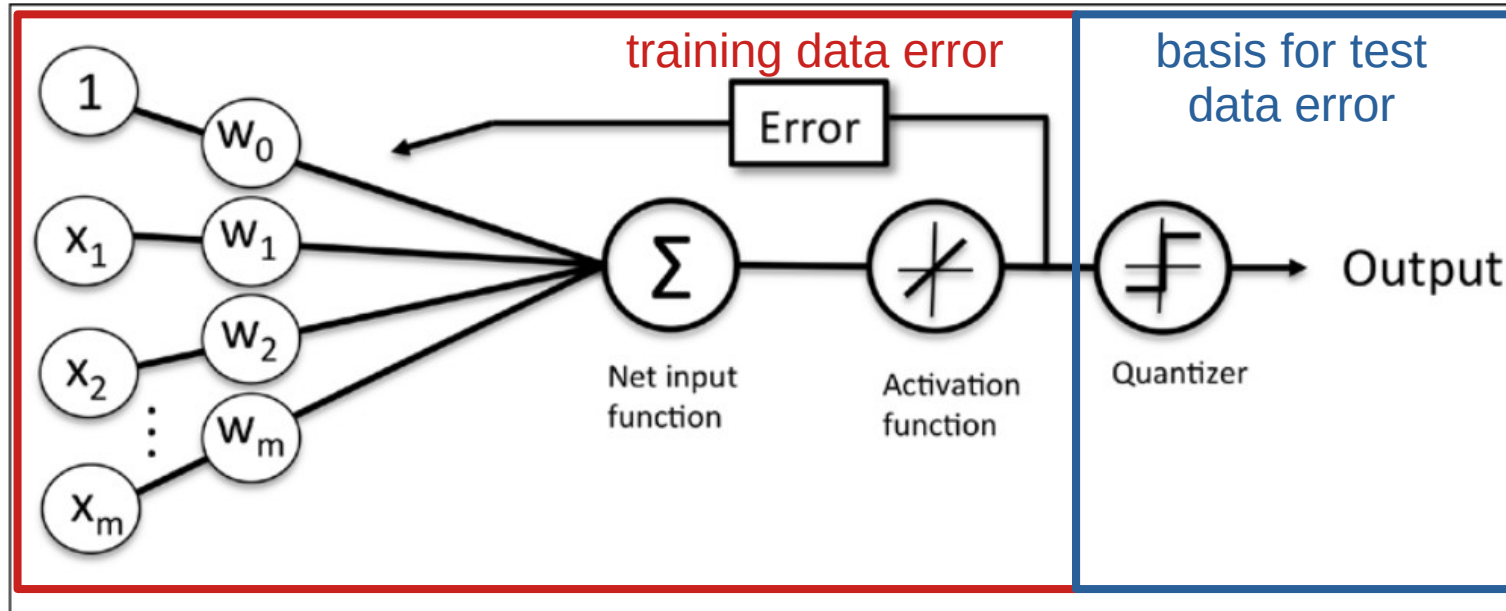
Summary

- We can build simple classification tools using scikit-learn
 - These can give us decision boundaries
 - That we can apply to new data (we haven't done that yet)
- We need to define cost functions
 - These can be conceptually separated from prediction functions (remember ADALINE)
- We can use linear regression to do continuous predictions

Recap

- We have *feature* vectors, $x^{(i)}$, for which we try to find the optimal weights, w .
 - Similar to $X\beta$
- The optimal w 's for the are the ones that minimise the error
 - Two kinds of error:
 - Perceptron
 - training data error: n misclassifications on training data
 - test data error: n misclassifications when generalised to test data
 - ADALINE
 - training data error: sum of squared errors on training data
 - test data error: n misclassifications when generalised to test data
 - Linear regression
 - training data error: sum of squared errors on training data
 - test data error: mean of squared errors when generalised to test data
- Our primary outcome is the test data error

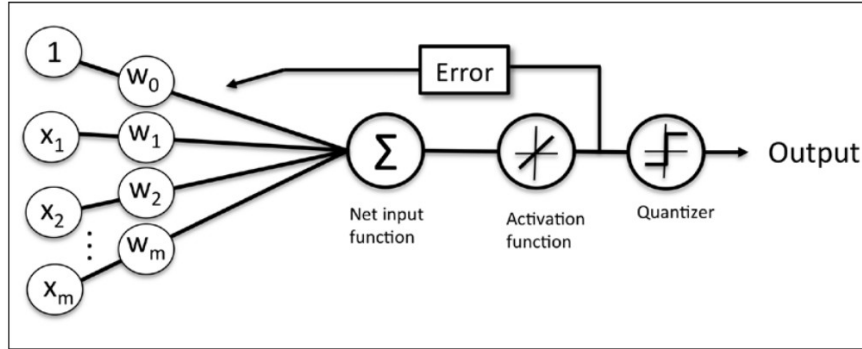
Recap



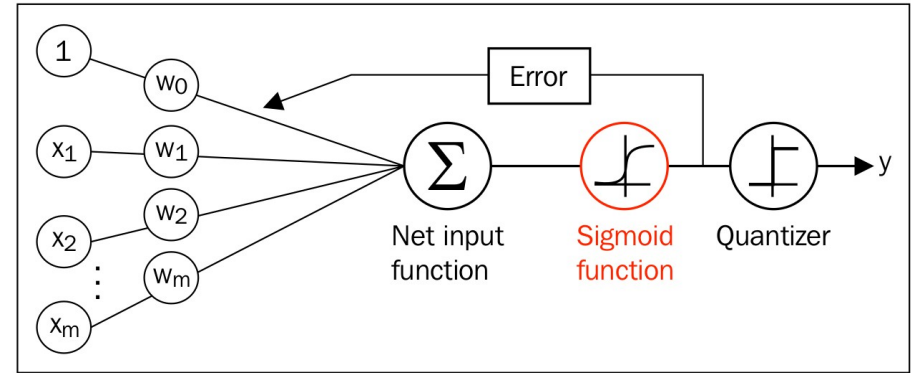
F
e
a
t
u
r
e
s

w
e
i
g
h
t
s

Moving to logistic regression



(p. 33: Raschka,
2015)



(p. 58: Raschka,
2015)

Learning goals and outline

Linear and logistic regression revisited (machine learning)

- 1) Understanding how logistic regression can be adapted to a classification framework
- 2) Understanding how gradient descent can provide a common framework across regression fits

WHY LOGISTIC REGRESSION?

Let's have a look at an iris flower

se·pal 🔊 (sē'pəl)

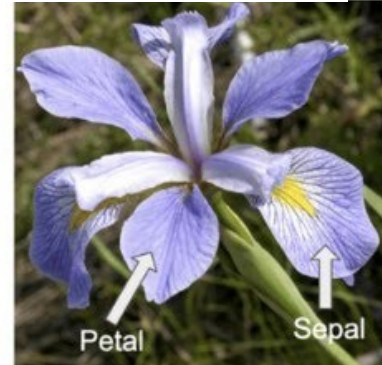
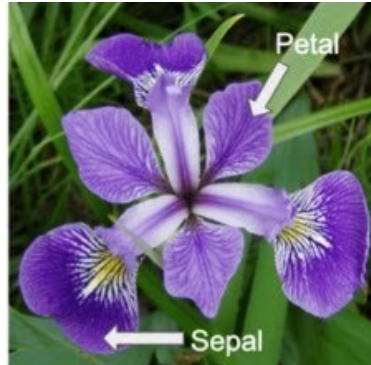
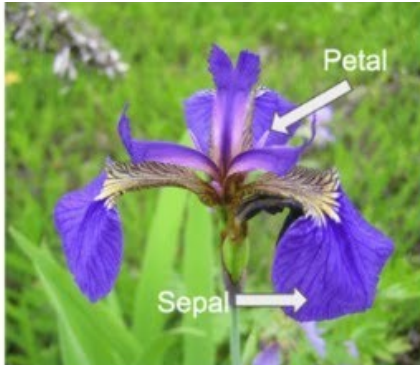
n.

One of the usually green leaflike structures composing the outermost part of a flower. Sepals often enclose and protect the bud and may remain after the fruit forms.

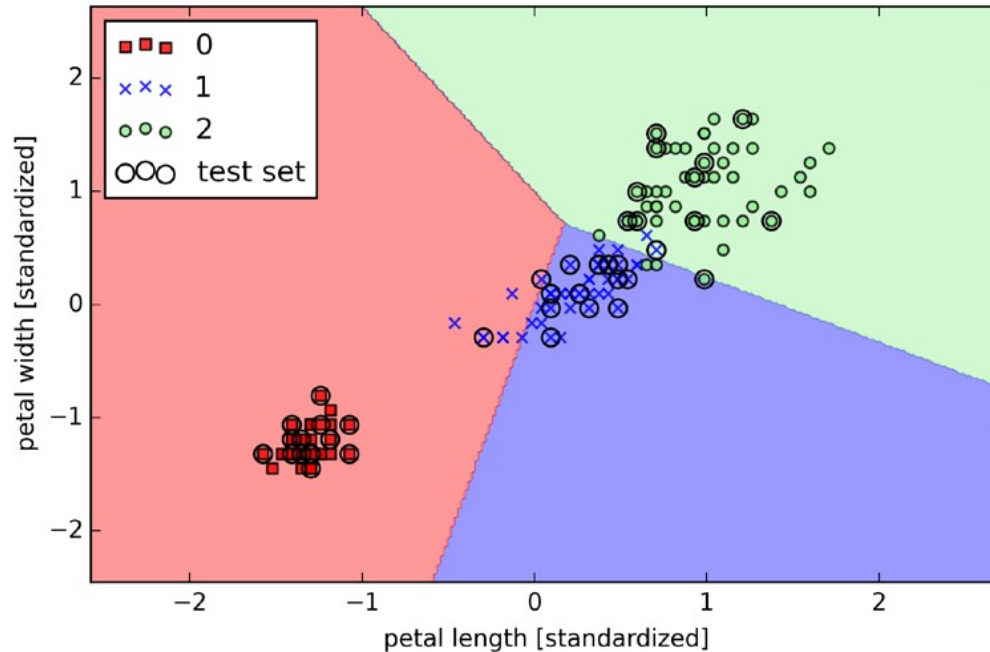
pet·al 🔊 (pět'l)

n.

One of the often brightly colored parts of a flower immediately surrounding the reproductive organs; a division of the corolla. —



The problem (Perceptron)



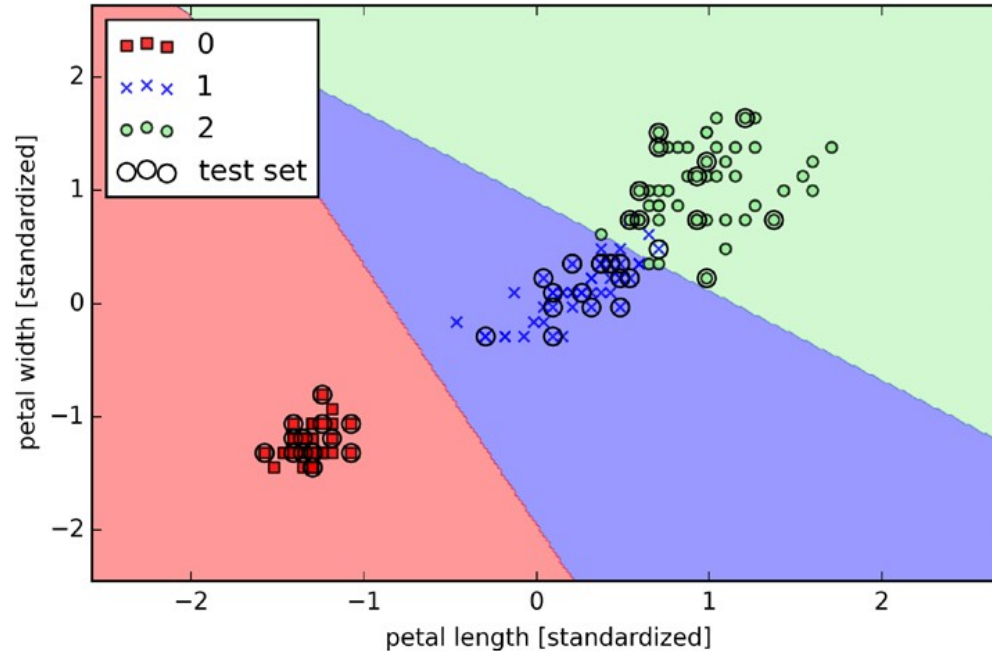
(p. 55: Raschka, 2015)

Not linearly separable → *never converges*

WANTED: an algorithm that converges
with linearly separable regions that
minimise the number of errors made

Something like this

LOGISTIC REGRESSION

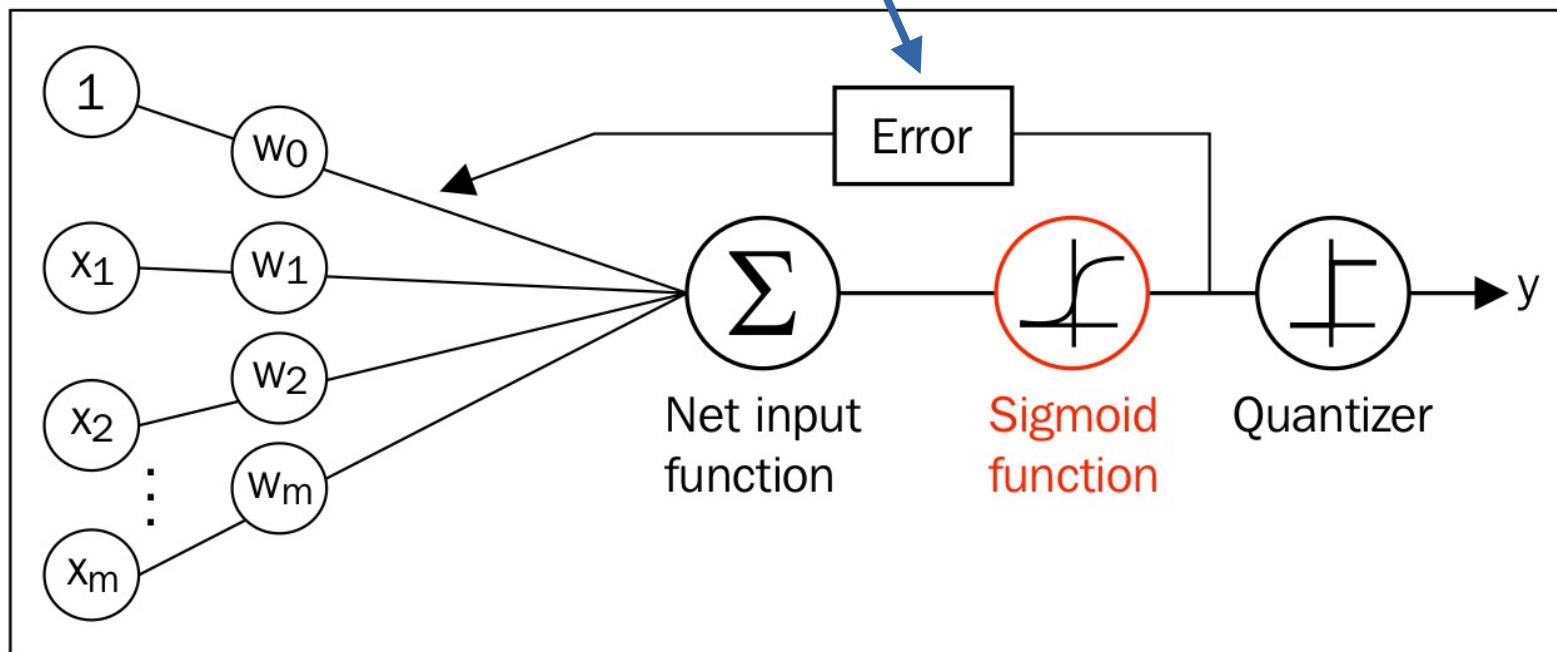


(p. 63: Raschka, 2015)

Separates flowers, while keeping errors at a minimum

What's our cost function, $J(w)$?

i.e. how do we calculate this?



Updating weights

ADALINE fit

$$\Delta \mathbf{w} = -\eta \nabla J(\mathbf{w})$$

$\Delta \mathbf{w}$: change in $\mathbf{w}$

η : learning rate

$\nabla J(\mathbf{w})$: gradient of cost function $J(\mathbf{w})$

$$J(\mathbf{w})_{\text{LINEAR}} = \left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 \right) / 2$$

$$J(\mathbf{w})_{\text{LOGISTIC}} = - \sum_{i=1}^n y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$$

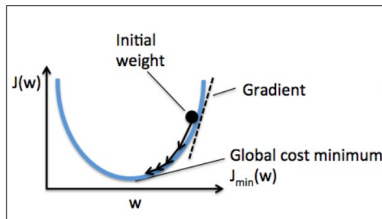
```
def fit(self, X, y):
    """ Fit training data.

    Parameters
    -----
    X : {array-like}, shape = [n_samples, n_features]
        Training vectors, where n_samples
        is the number of samples and
        n_features is the number of features.
    y : array-like, shape = [n_samples]
        Target values.

    Returns
    -----
    self : object

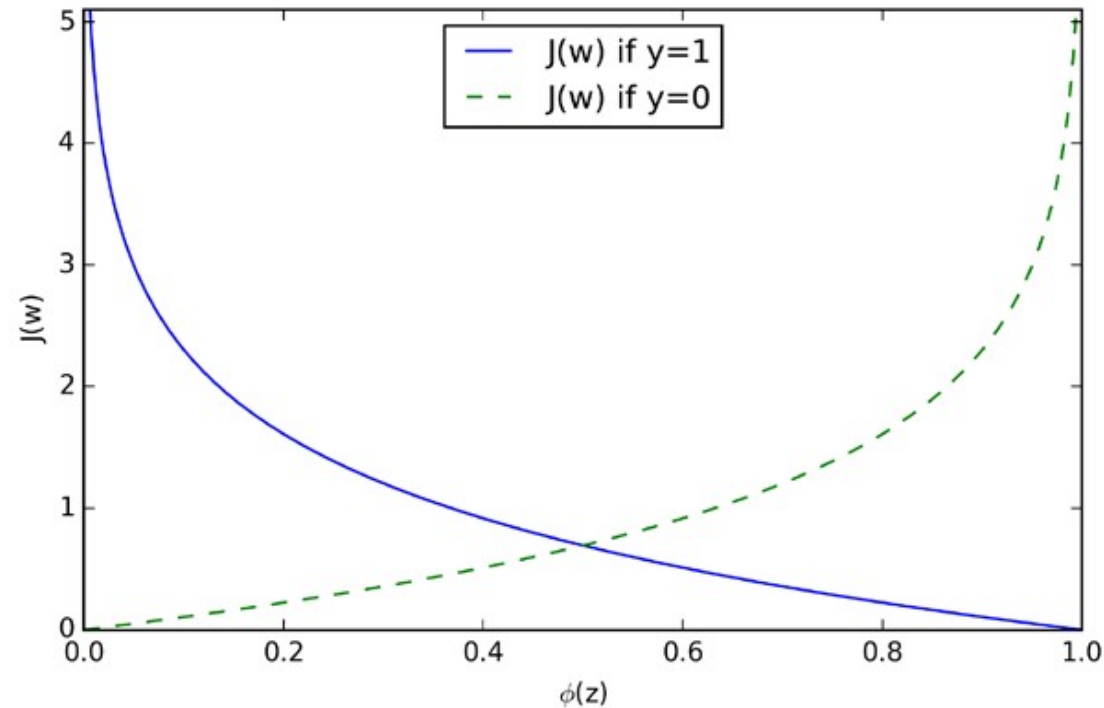
    """
    self.w_ = np.zeros(1 + X.shape[1])
    self.cost_ = []

    for i in range(self.n_iter):
        output = self.net_input(X)
        errors = (y - output)
        self.w_[1:] += self.eta * X.T.dot(errors)
        self.w_[0] += self.eta * errors.sum()
        cost = (errors**2).sum() / 2.0
        self.cost_.append(cost)
    return self
```



Where have
you seen this
before?

Cost function for one instance



L1 and L2 regularisation

$$J(\mathbf{w})_{L_1} = -\sum_{i=1}^n \mathbf{y}_i \log(\phi(\mathbf{z}_i)) + (1 - \mathbf{y}_i) \log(1 - \phi(\mathbf{z}_i)) + \lambda \|\mathbf{w}\|_1$$

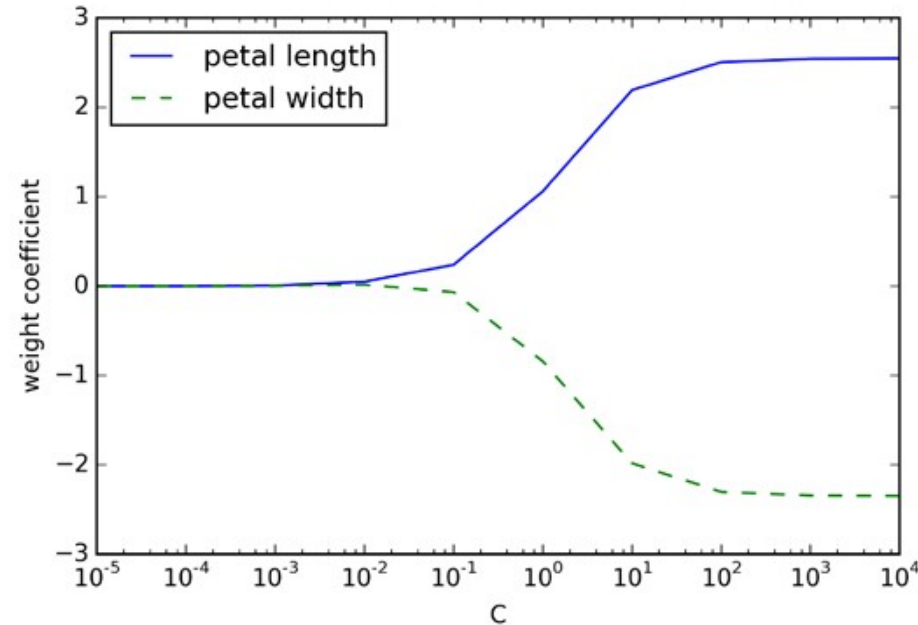
$$J(\mathbf{w})_{L_2} = -\sum_{i=1}^n \mathbf{y}_i \log(\phi(\mathbf{z}_i)) + (1 - \mathbf{y}_i) \log(1 - \phi(\mathbf{z}_i)) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

NB! Regularisation in *sklearn.linear_model.LogisticRegression* is controlled by *C*

$$C = \frac{1}{\lambda}$$

L2 regularisation

$$C = \frac{1}{\lambda}$$



```
logR_l2 = LogisticRegression(penalty='l2', C=10)
```

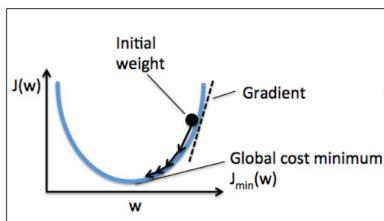
Updating weights – the general case

proof on p. 64

$$\Delta W_j = -\eta \frac{\delta J}{\delta W_j} = \eta \sum_{i=1}^n (y_i - \phi(\mathbf{z}_i)) x_j$$

$$\phi(\mathbf{z})_{\text{LINEAR}} = \mathbf{z}$$

$$\phi(\mathbf{z})_{\text{LOGISTIC}} = \frac{1}{(1 + e^{-z})}$$



```
def fit(self, X, y):
    """ Fit training data.

    Parameters
    -----
    X : {array-like}, shape = [n_samples, n_features]
        Training vectors, where n_samples
        is the number of samples and
        n_features is the number of features.
    y : array-like, shape = [n_samples]
        Target values.

    Returns
    -----
    self : object
    """
    self.w_ = np.zeros(1 + X.shape[1])
    self.cost_ = []

    for i in range(self.n_iter):
        output = self.net_input(X)
        errors = (y - output)
        self.w_[1:] += self.eta * X.T.dot(errors)
        self.w_[0] += self.eta * errors.sum()
        cost = (errors**2).sum() / 2.0
        self.cost_.append(cost)
    return self
```

ADALINE
fit

Exercise in class

```
class LogisticRegressionGD:
    def __init__(self, eta=0.01, n_iterations=1000, tol=1e-4):
        self.eta = eta
        self.n_iterations = n_iterations
        self.w_ = None
        self.cost_ = []
        self.tol = tol
        self.n_iter_ = 0

    def fit(self, X, y):

        X = np.insert(X, 0, 1, axis=1) # adding intercept term
        self.w_ = np.zeros(X.shape[1])
        # Gradient Descent
        for _ in range(self.n_iterations):
            self.n_iter_ += 1
            # Calculate linear combination of features and weights
            linear_model = None ## WHAT IS SUPPOSED TO BE HERE?

            # Apply sigmoid function to get probabilities
            yhat = sigmoid(linear_model) ## WHAT IS SUPPOSED TO BE HERE?

            self.cost_.append(cost_function(y, yhat))

            # Update weights using the gradient
            self.w_ += None ## WHAT IS SUPPOSED TO BE HERE?
            if self.n_iter_ > 1: ## we need at least two observations
                diff = None ## WHAT IS SUPPOSED TO BE HERE?
                # print(diff)
                if diff < self.tol:
                    break

    def predict(self, X):
        pass ## put in your own quantizer function if you feel up for it
```

```
X, y = load_iris(return_X_y=True)
X = X[y < 2, 0:3:2] ## petal length and sepal length
y = y[y < 2]

logreg = LogisticRegressionGD(eta=0.001, n_iterations=10000, tol=1e-4)
logreg.fit(X, y)

plt.figure()
plt.xlabel('Iteration (#)')
plt.ylabel('Cost (J(w))')
plt.title('Converged at iteration: ' + str(logreg.n_iter_))
plt.plot(logreg.cost_)
plt.show()

print(logreg.predict(X))
print(logreg.predict(X) == y)
```

Find code on Brightspace

Interim summary

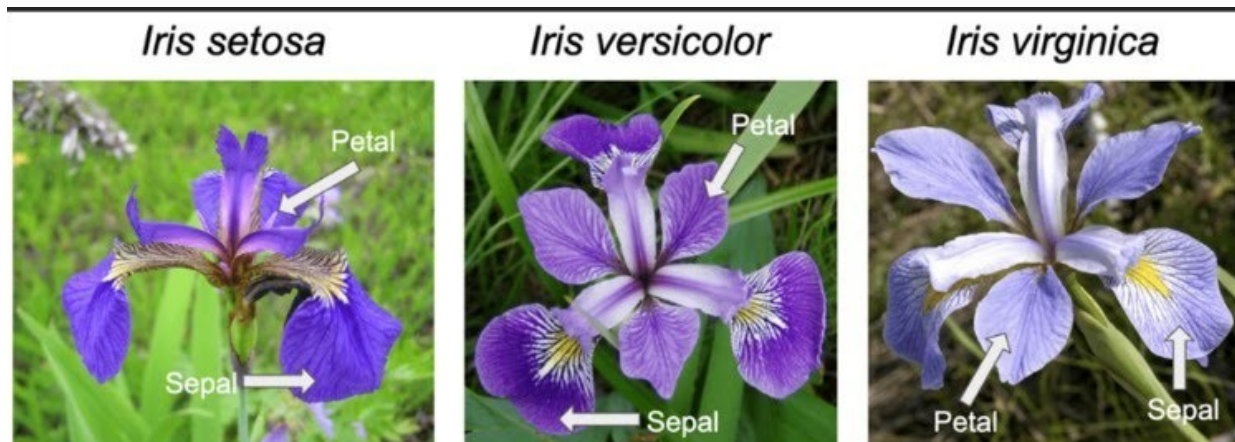
- Gradient descent can be put into a common framework
 - $\Delta w_j = -\eta \frac{\delta J}{\delta w_j} = \eta \sum_{i=1}^n (\mathbf{y}_i - \phi(\mathbf{z}_i)) x_j$
 - we would thus also be able to do Poisson machine learning (not covered here), and any other transformation that would be meaningful
- Now, we need to move beyond fitting and test

Steps of machine learning analysis

1. Selection of features.
2. Choosing a performance metric.
3. Choosing a classifier and optimization algorithm.
4. Evaluating the performance of the model.
5. Tuning the algorithm.

(p. 50: Raschka,
2015)

1. Selection of features, i.e. X



<https://i.ytimg.com/vi/E8NrI9QLNq8/maxresdefault.jpg>

It is a good idea to standardise these

$$x_{std}^{(i)} = \frac{x^{(i)} - \mu_x}{\sigma_x}$$

2. Choosing a performance metric

E.g. Accuracy of classification or Reduction of error

3. Choosing a classifier and an optimization algorithm

Logistic regression and gradient descent

Linear regression and gradient descent

4. Evaluating the performance of the model

Perform out-of-sample testing

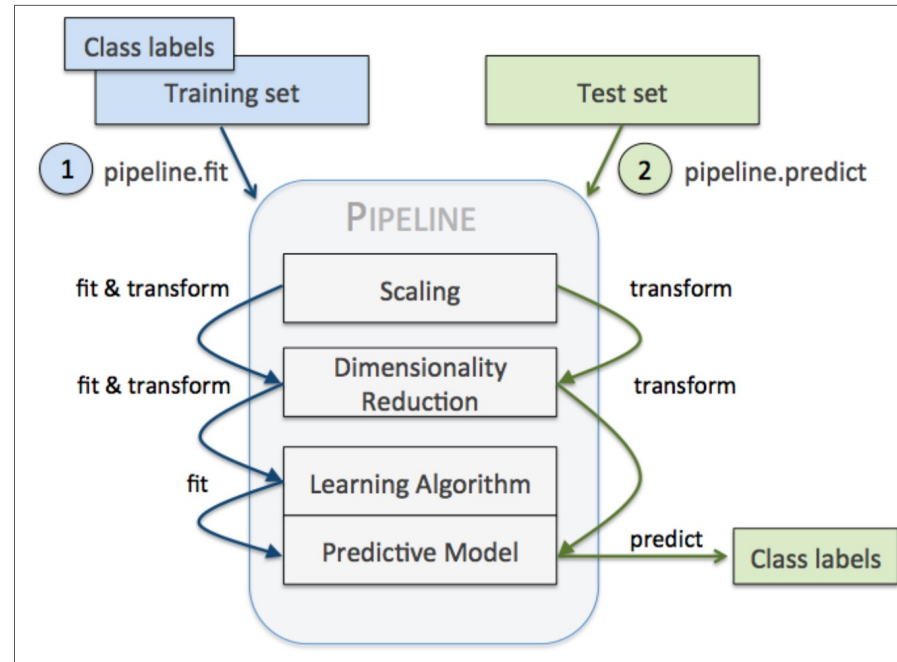
Do this in a cross-validated manner where the data, several times, is partitioned into training and test data sets, to minimise bias.

5. Tuning our model

If our model performs better on the training data than on the test data, we are likely to be overfitting. Regularisation could be a solution. We can do e.g. *grid search* or *randomised search* (next week) to find the optimal C-values in logistic and linear regression

We could also try to reduced the number of features in the model and only retain the ones explaining the most variance, thereby reducing the risk of overfitting, e.g. by using *Principled component analysis* (in two weeks)

General structure



(p. 172: Raschka, 2015)

Can be implemented in *scikit-learn* (in four weeks)

```
from sklearn.pipeline import Pipeline
```

```
pipe = Pipeline(steps=[("scaler", scaler), ("pca", pca), ("logistic", logistic)])
```

Live coding

RUNNING LOGISTIC REGRESSION

Summary

- Our standard regression methods can be set up for prediction
- Scikit-learn can help us set the whole pipeline for classification
- Support Vector Machines are very similar to logistic regression, but may generalise better

Learning goals and outline

Linear and logistic regression revisited (machine learning)

- 1) Understanding how linear and logistic regression can be adapted to a classification framework
- 2) Understanding how gradient descent can provide a common framework across regression fits

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

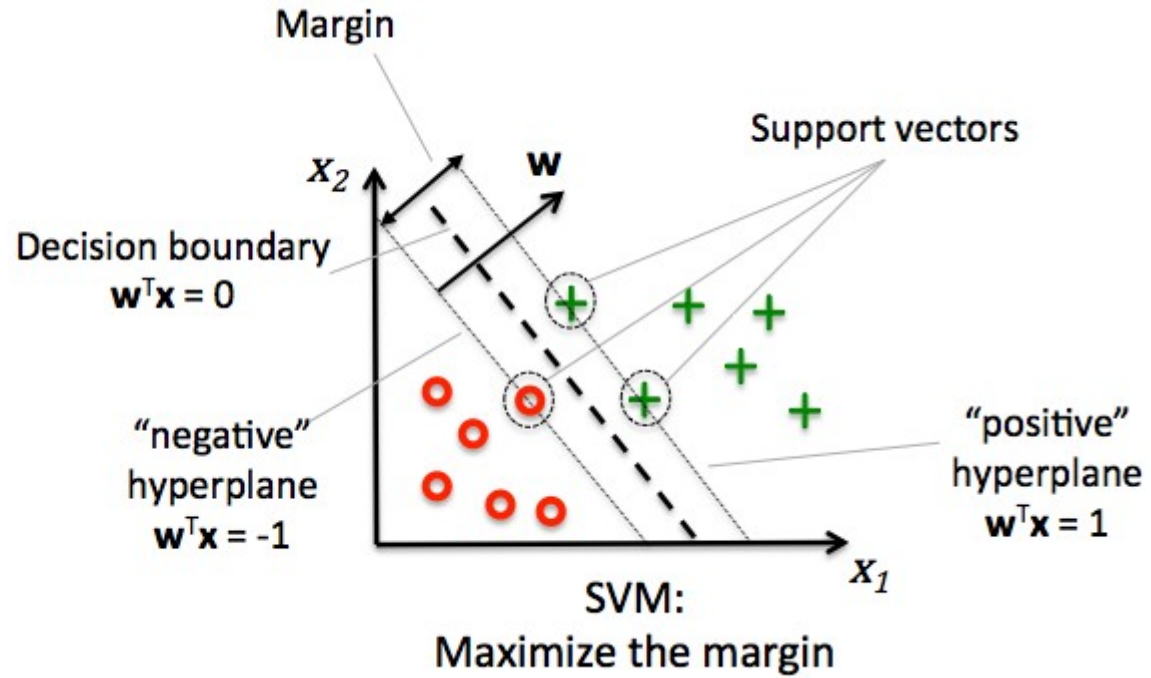
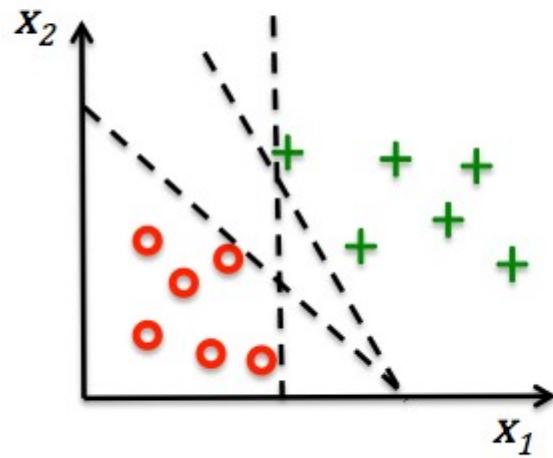
Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Next time

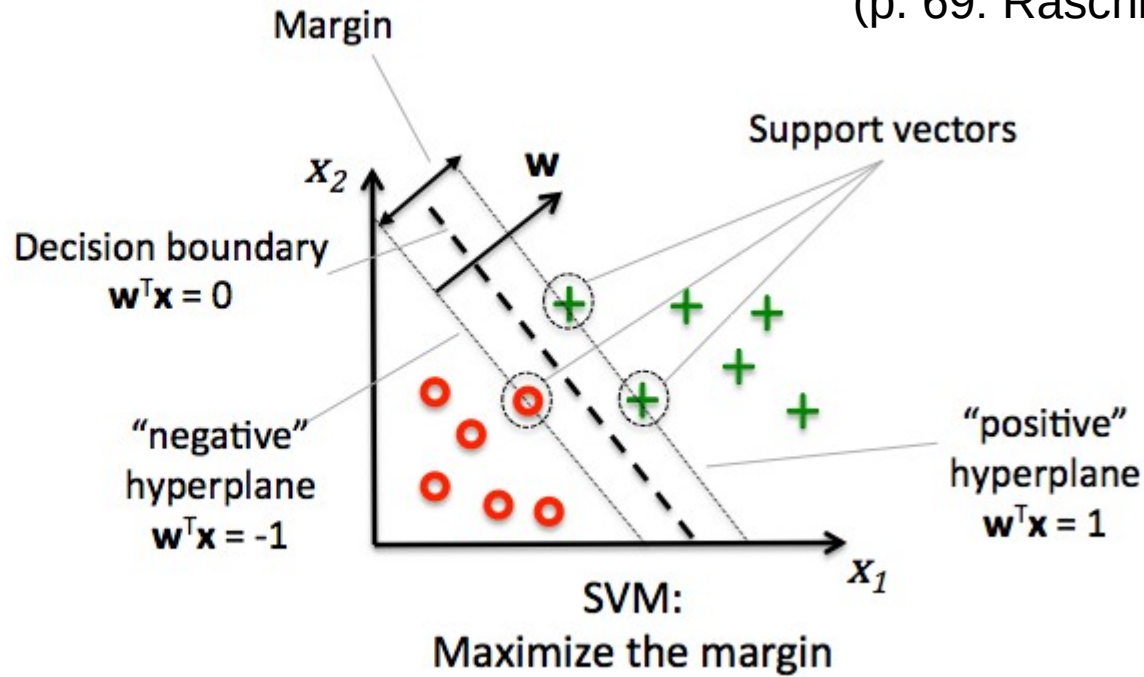
- Doing search for the best hyperparameters
- Confusion matrices
- Validation Accuracy

Extra slides on Support Vector Machines

SUPPORT VECTOR MACHINES



(p. 69: Raschka, 2015)



$$w_0 + \mathbf{w}^T \mathbf{x}_{\text{pos}} = 1 \quad (1)$$

$$w_0 + \mathbf{w}^T \mathbf{x}_{\text{neg}} = -1 \quad (2)$$

QUESTION: What is the subtraction of equation (2) from equation (1) equal to on the figure?

Subtraction

$$\mathbf{w}^T (\mathbf{x}_{\text{pos}} - \mathbf{x}_{\text{neg}}) = 2$$

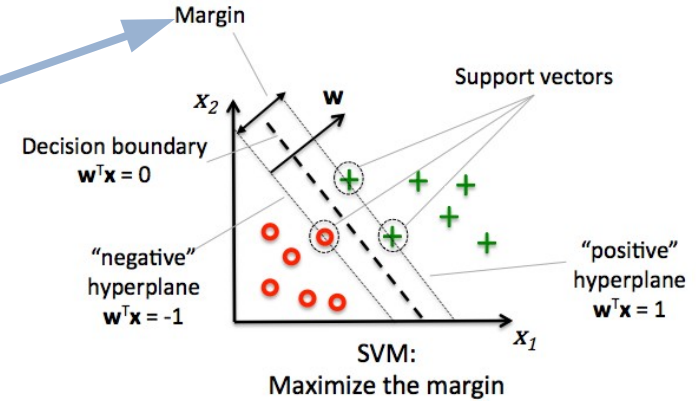
normalizing by $\mathbf{w}$'s length: $\|\mathbf{w}\| = \sqrt{\left(\sum_{j=1}^m w_j^2\right)}$

$$\dots \text{ we get: } \mathbf{w}^T \frac{(\mathbf{x}_{\text{pos}} - \mathbf{x}_{\text{neg}})}{\|\mathbf{w}\|} = \frac{2}{\|\mathbf{w}\|}$$

This will optimise the width of the margin

So we want to maximize: $\frac{2}{\|\mathbf{w}\|}$
under the constraints:

$$w_0 + \mathbf{w}^T \mathbf{x}^{(i)} \geq 1 \text{ if } y^{(i)} = 1$$
$$w_0 + \mathbf{w}^T \mathbf{x}^{(i)} < -1 \text{ if } y^{(i)} = -1$$



(p. 69: Raschka, 2015)

This faces a problem that the
Perceptron also faces

Which?

SOLUTION: introducing a *slack*
variable ξ (x_i)

Introducing ξ

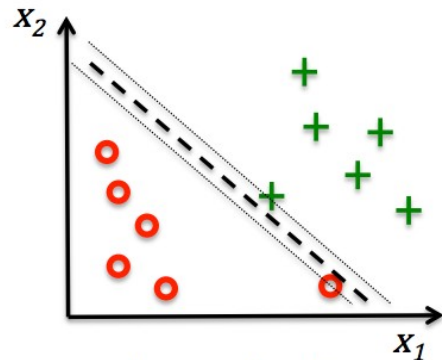
$$w_0 + \mathbf{w}^T \mathbf{x}_{\text{pos}} = 1 - \xi^{(i)}$$

$$w_0 + \mathbf{w}^T \mathbf{x}_{\text{neg}} = -1 + \xi^{(i)}$$

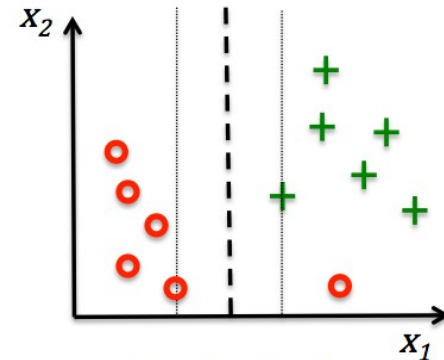
now we need to minimize: $\frac{1}{2} \|\mathbf{w}\|^2 + C \left(\sum_i^n \xi^{(i)} \right)$ (p. 72: Raschka, 2015)

to maximise the margin

Large C : narrow margin –
penalises errors harshly
Small C : wide margin –
penalises errors mildly

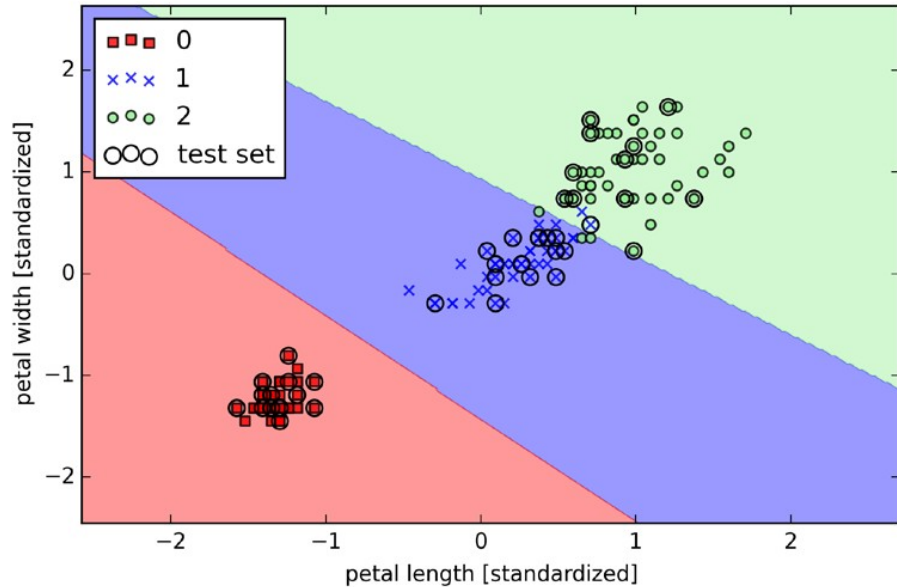


Large value for
parameter C

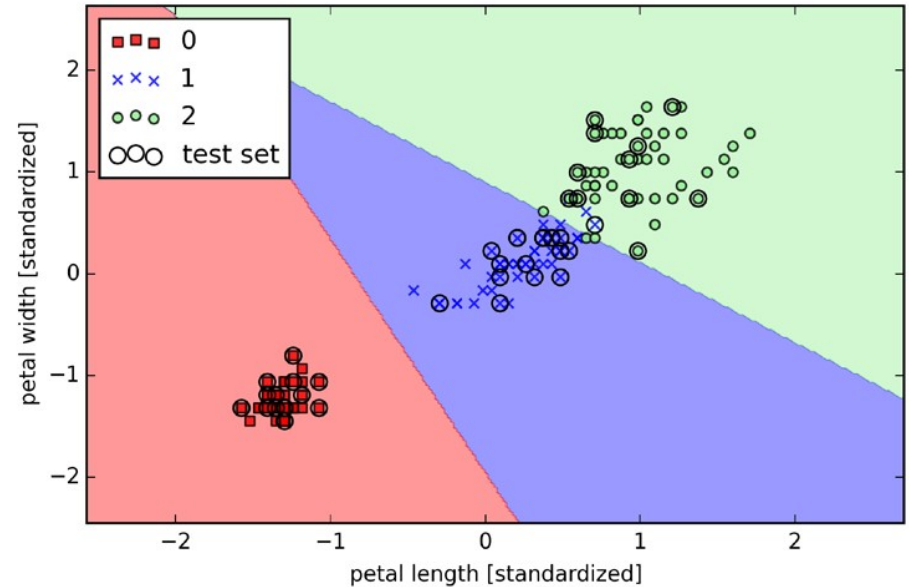


Small value for
parameter C

Comparison to logistic regression



Support Vector Machine (p.73,
Raschka 2015)



Logistic regression (p. 63,
Raschka 2015)

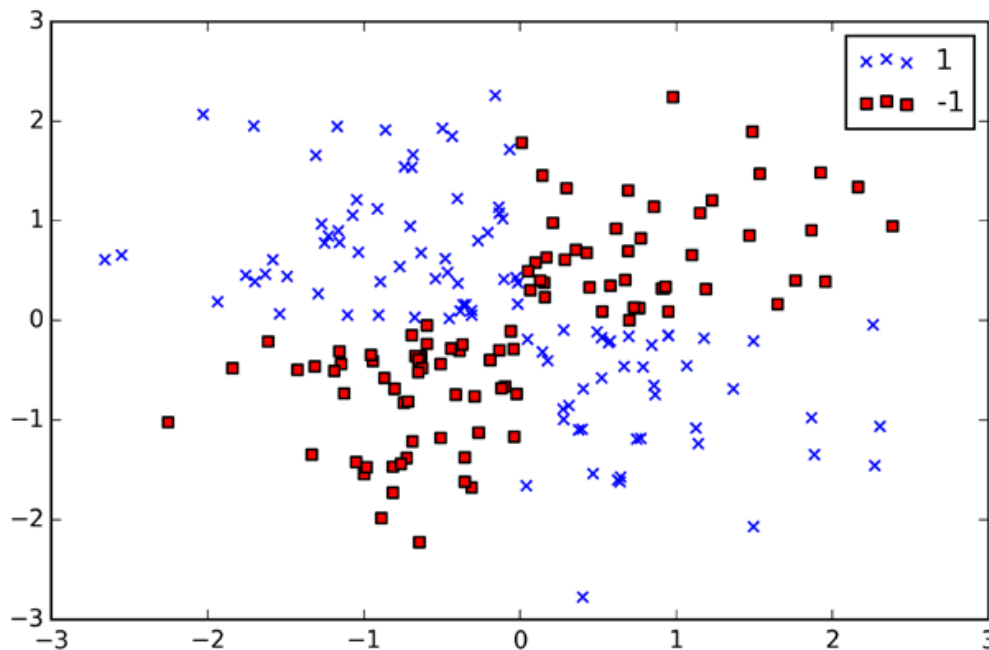


Logistic regression versus SVM

In practical classification tasks, linear logistic regression and linear SVMs often yield very similar results. Logistic regression tries to maximize the conditional likelihoods of the training data, which makes it more prone to outliers than SVMs. The SVMs mostly care about the points that are closest to the decision boundary (support vectors). On the other hand, logistic regression has the advantage that it is a simpler model that can be implemented more easily. Furthermore, logistic regression models can be easily updated, which is attractive when working with streaming data.

(p. 74: Raschka, 2015)

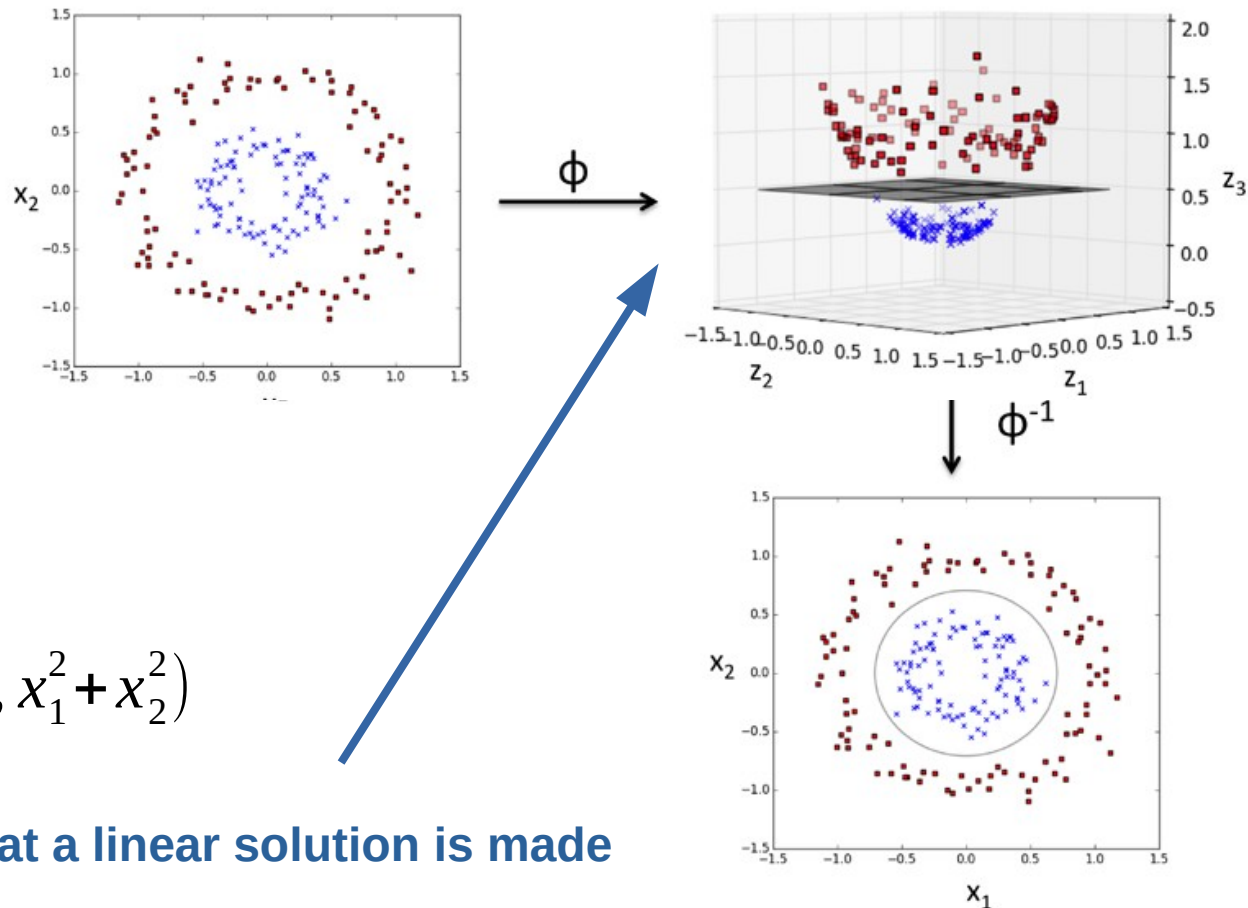
A brief look at a non-linear problem



(p. 75: Raschka, 2015)

Map onto higher-dimensional space

$$\phi(\cdot)$$



$$\phi(x_1, x_2) = (z_1, z_2, z_3) = (x_1, x_2, x_1^2 + x_2^2)$$

Note that a linear solution is made

$$\phi(x_1, x_2) = (z_1, z_2, z_3) = (x_1, x_2, x_1^2 + x_2^2)$$

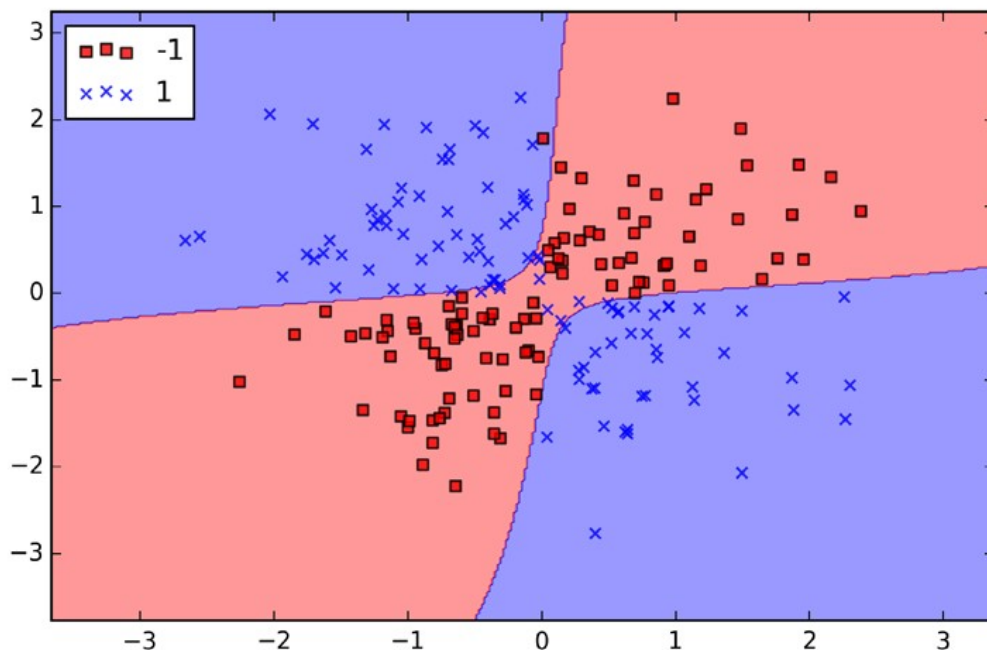
Creating the higher dimensions
can be computationally expensive

$$k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \phi(\mathbf{x}^{(i)})^T \phi(\mathbf{x}^{(j)})$$

$$k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = e^{(-\gamma \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2)}$$

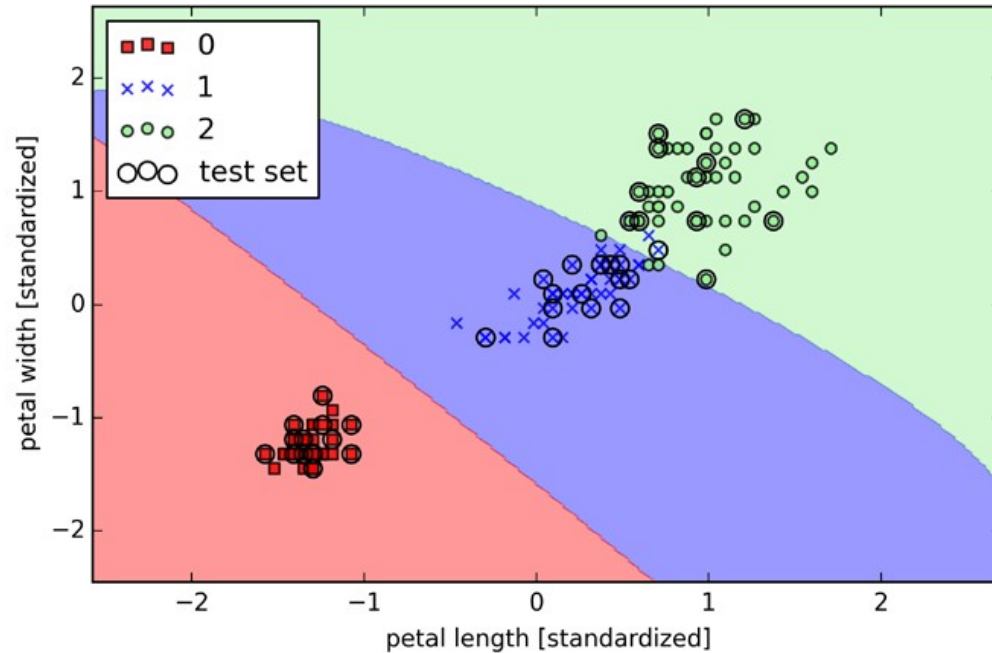
$$(\gamma = \frac{1}{2\sigma^2}, \text{ also called the precision})$$

Non-linear decision boundaries



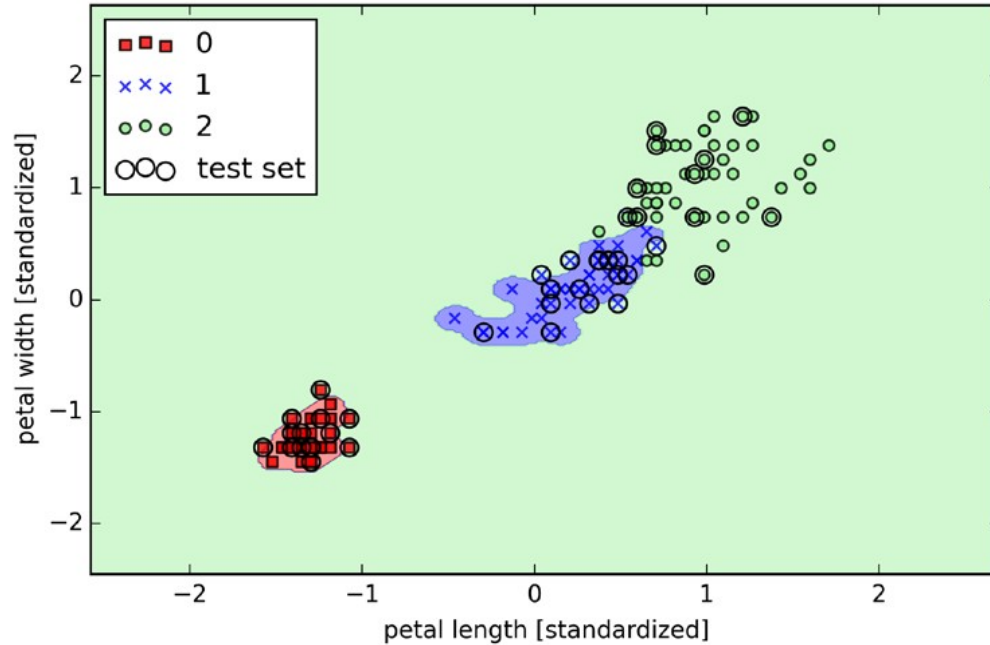
(p. 78: Raschka, 2015)

Low γ - soft boundary



(p. 79: Raschka, 2015)

High γ - tight boundary



(p. 80: Raschka, 2015)